콘텐츠 주소 지정 시스템

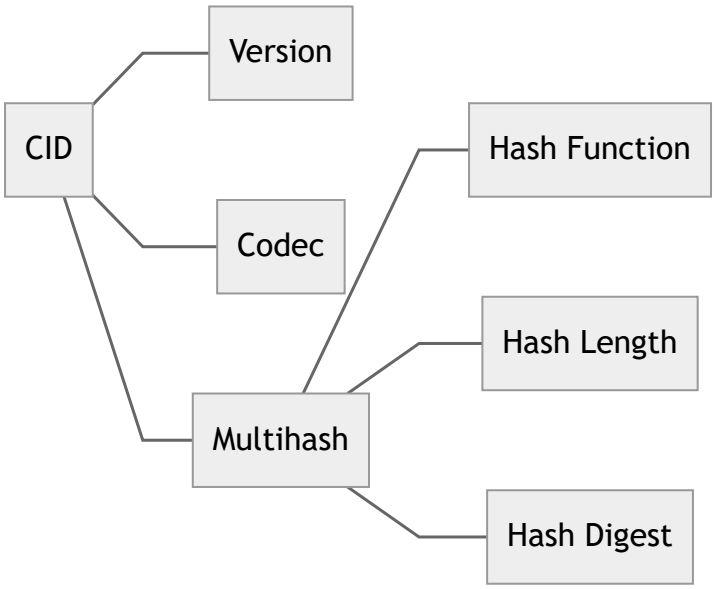
개요

IPFS의 콘텐츠 주소 지정 시스템은 데이터의 내용을 기반으로 고유한 식별자를 생성하는 핵심 메커니즘입니다. 이 시스템은 CID(Content Identifier)를 통해 데이터의 무결성을 보장하고, 중복 제거 및 효율적인 데이터 검색을 가능하게 합니다.

CID (Content Identifier) 구조

CID는 IPFS에서 모든 콘텐츠를 식별하는 자체 설명적(self-describing) 식별자입니다.

CID 구성 요소



CID 버전

CIDv0

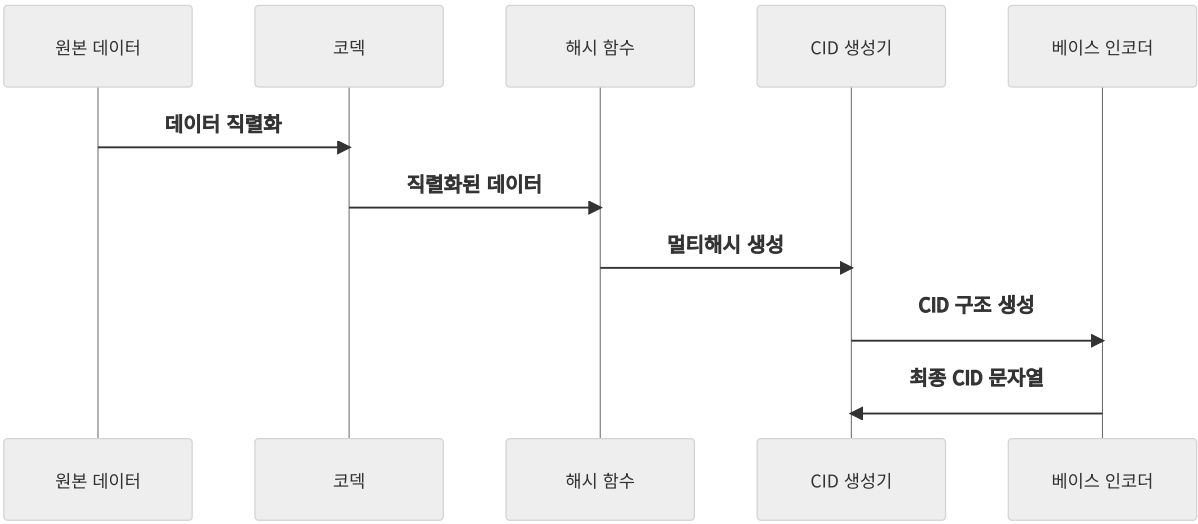
- **형식**: Base58btc로 인코딩된 SHA-256 해시

- **특징**:
- 항상 'Qm'으로 시작
- DAG-PB 코덱만 지원
- SHA-256 해시 함수만 사용
- **예시**: `QmYwAPJzv5CZsnA625s3Xf2nemtYgPpHdWEz79ojWnPbdG`

CIDv1

- **형식**: 멀티베이스로 인코딩된 구조화된 식별자
- **특징**:
- 다양한 코덱 지원
- 다양한 해시 함수 지원
- 다양한 베이스 인코딩 지원
- **예시**: `bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3oclgtqy55fbzdi`

CID 생성 과정



해시 함수 및 멀티해시

지원되는 해시 함수

IPFS는 다양한 해시 함수를 지원하며, 각각 고유한 특성을 가집니다:

SHA-256 (기본값)

- **코드**: 0x12
- **길이**: 32바이트
- **특징**:
- 가장 널리 사용되는 해시 함수
- 높은 보안성과 안정성
- 비트코인에서도 사용
- **사용 사례**: 일반적인 모든 용도

SHA-512

- **코드**: 0x13
- **길이**: 64바이트
- **특징**:
- SHA-256보다 더 긴 해시 길이
- 더 높은 보안 수준
- **사용 사례**: 높은 보안이 필요한 경우

Blake2b

- **코드**: 0xb220
- **길이**: 가변 (최대 64바이트)
- **특징**:
- SHA-3보다 빠른 성능
- 가변 길이 출력 지원
- **사용 사례**: 성능이 중요한 환경

Blake2s

- **코드**: 0xb240
- **길이**: 가변 (최대 32바이트)
- **특징**:
- Blake2b의 32비트 최적화 버전
- 더 작은 메모리 사용량
- **사용 사례**: 리소스가 제한된 환경

멀티해시 형식

멀티해시는 해시 함수와 해시 길이 정보를 포함하는 자체 설명적 해시 형식입니다:

```
<hash-function-code><hash-length><hash-digest>
```

멀티해시 구성 예시

```
// SHA-256 해시의 멀티해시 구성
hashFunction := 0x12  // SHA-256 코드
hashLength := 0x20  // 32바이트 길이
hashDigest := [32]byte{...} // 실제 해시 값

multihash := append([]byte{hashFunction, hashLength}, hashDigest...)
```

CID 생성 및 해석 과정

CID 생성 단계별 과정

1단계: 데이터 준비

```
// 원본 데이터
data := []byte("Hello, IPFS!")
```

2단계: 코덱 적용

```
// DAG-PB 코덱으로 직렬화
```

```
serializedData := dagpb.Encode(data)
```

3단계: 해시 계산

```
// SHA-256 해시 계산
hasher := sha256.New()
hasher.Write(serializedData)
hashDigest := hasher.Sum(nil)
```

4단계: 멀티해시 생성

```
// 멀티해시 구성
multihash := multihash.Encode(hashDigest, multihash.SHA2_256)
```

5단계: CID 구성

```
// CIDv1 생성
cidV1 := cid.NewCidV1(cid.DagProtobuf, multihash)
```

6단계: 베이스 인코딩

```
// Base32 인코딩
cidString := cidV1.String() // "bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3ocl
```

CID 해석 과정

CID 파싱

```
func ParseCID(cidString string) (*CIDInfo, error) {
// CID 디코딩
c, err := cid.Decode(cidString)
if err != nil {
return nil, err
}
// 버전 확인
version := c.Version()
// 코덱 추출
codec := c.Type()
// 멀티해시 추출
mh := c.Hash()
// 해시 함수 코드 추출
hashCode, err := multihash.GetCode(mh)
if err != nil {
return nil, err
}
// 해시 길이 추출
hashLength, err := multihash.GetLength(mh)
if err != nil {
return nil, err
```

```
}

// 해시 다이제스트 추출

hashDigest, err := multihash.GetDigest(mh)

if  err != nil {

return  nil, err

}

return &CIDInfo{

Version: version,

Codec: codec,

HashCode: hashCode,

HashLength: hashLength,

HashDigest: hashDigest,

}, nil

}
```

콘텐츠 검증 및 무결성 보장

해시 기반 검증

IPFS는 CID를 통해 데이터의 무결성을 자동으로 검증합니다:

```
func  VerifyContent(cid cid.Cid, data []byte) error {

// 데이터로부터 해시 계산

expectedHash := calculateHash(data, cid.Hash().Code())
```

```
// CID의 해시와 비교
if !bytes.Equal(expectedHash, cid.Hash().Digest()) {
return errors.New("content verification failed: hash mismatch")
}
return nil
}
```

검증 프로세스

1. **데이터 수신**: 네트워크에서 블록 데이터 수신
2. **해시 계산**: 수신된 데이터의 해시 계산
3. **CID 비교**: 계산된 해시와 CID의 해시 비교
4. **검증 결과**: 일치하면 유효, 불일치하면 오류

보안 고려사항

해시 함수 보안

- **충돌 저항성**: 서로 다른 입력이 같은 해시를 생성하기 어려움
- **역상 저항성**: 해시로부터 원본 데이터를 찾기 어려움
- **제2 역상 저항성**: 주어진 입력과 같은 해시를 가진 다른 입력을 찾기 어려움

허용된 해시 함수

IPFS는 보안상 안전한 해시 함수만 허용합니다:

```
// 허용된 해시 함수 목록
var DefaultAllowlist = verifcid.Allowlist{
```

```
Codes: map[uint64]bool{

mh.SHA2_256: true,

mh.SHA2_512: true,

mh.SHA3_224: true,

mh.SHA3_256: true,

mh.SHA3_384: true,

mh.SHA3_512: true,

mh.BLAKE2B_MIN + 31: true, // Blake2b-256

mh.BLAKE2S_MIN + 31: true, // Blake2s-256

},

}
```

베이스 인코딩

지원되는 베이스 인코딩

Base58btc (CIDv0 기본값)

- **접두사**: 없음
- **문자 집합**: `123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz`
- **특징**: 혼동하기 쉬운 문자(0, O, I, l) 제외

Base32 (CIDv1 기본값)

- **접두사**: `b`
- **문자 집합**: `abcdefghijklmnopqrstuvwxyz234567`

- **특징**: 대소문자 구분 없음, URL 안전

Base16 (Hex)

- **접두사**: `f`
- **문자 집합**: `0123456789abcdef`
- **특징**: 가장 간단한 형태, 디버깅에 유용

Base64

- **접두사**: `m`
- **문자 집합**: `ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/`
- **특징**: 높은 정보 밀도

베이스 변환

```
func ConvertCIDBase(cidString string, targetBase multibase.Encoding) (string,

// CID 파싱

c, err := cid.Decode(cidString)

if err != nil {

return "", err

}

// 새로운 베이스로 인코딩

encoder, err := multibase.NewEncoder(targetBase)

if err != nil {

return "", err

}
```

```
return c.StringOfBase(targetBase)
}
```

실제 사용 예시

블록 저장 시 CID 생성

```
func StoreBlock(data []byte, codec uint64, hashFunc uint64) (cid.Cid, error)
// 해시 계산
hasher, err := multihash.GetHasher(hashFunc)
if err != nil {
return cid.Undef, err
}
hasher.Write(data)
hash := hasher.Sum(nil)
// 멀티해시 생성
mh, err := multihash.Encode(hash, hashFunc)
if err != nil {
return cid.Undef, err
}
// CID 생성
return cid.NewCidV1(codec, mh), nil
```

```
}
```

CID 기반 블록 검색

```
func RetrieveBlock(c cid.Cid, blockstore Blockstore) ([]byte, error) {
// 블록스토어에서 블록 검색
block, err := blockstore.Get(c)
if err != nil {
return nil, err
}
// 무결성 검증
if err := VerifyContent(c, block.RawData()); err != nil {
return nil, err
}
return block.RawData(), nil
}
```

성능 최적화

해시 계산 최적화

- **하드웨어 가속**: AES-NI 등 하드웨어 가속 기능 활용
- **병렬 처리**: 큰 파일의 경우 청크 단위로 병렬 해시 계산
- **캐싱**: 자주 사용되는 해시 결과 캐싱

CID 처리 최적화

- **문자열 풀링**: CID 문자열 객체 재사용
- **바이너리 형태 사용**: 내부적으로 바이너리 형태로 CID 처리
- **배치 처리**: 여러 CID를 한 번에 처리

이러한 콘텐츠 주소 지정 시스템을 통해 IPFS는 데이터의 무결성을 보장하고, 효율적인 중복 제거 및 콘텐츠 검색을 제공합니다.